

Veri Yapıları Ve Algoritmalar

Online yıldırdı edu tr Ders durarı içerisinde ders içeriği mevcut

10:05 geçe ders başlıyor.

45 dk ders 15 dk ara

bu dersde not tutmak önemli!

2vi 2e

% 20

4 ödev

% 12

1 proje

% 8

Sinav

% 40

Devam zorunluluğu Yok!

Sınavlar algoritmik şekilde olacak
Gecmiş sınavlara dikkat et.

Lablarda bazen konu bazen uygulama bazende
ödev verilir ödev kontrolü yapılacaktır. 1-2 hafta
dışında aktif kullanılabılır.

Dersle gelmeden önce syllabus'a bak.

1. Hafta

Algorithm Design Criteria

- * Correctness
- * Easy to implement
- * Easy to understand
- * Easy to modify
- * Time efficiency
- * Space efficiency

Algorithm Presentation

- * flowchart
- * Pseudo Code
- * Programming Language
- * Dial presentation

Programming

- * Arsi C standard
- * Global Variables (Do not use)
- * Variable Names, function Names
- * Modular Programming
- * Dynamic Memory Allocation
- * Comment Lines

Program = Algorithm + Data Structures

Data Structures

- * Arrays
- * Linked Lists
- * Stack
- * Queue
- * Tree
- * Graphs
- * Heap

decrease and conquer

Search Algorithms

- * Linear $O(n)$
- * Binary search $\log_2 n$
- * Heatching or Hashing?
- * Jump search $O(\sqrt{n})$
- * Binary search tree
- * Interpolation search
- * Exponential search

Basic Operations

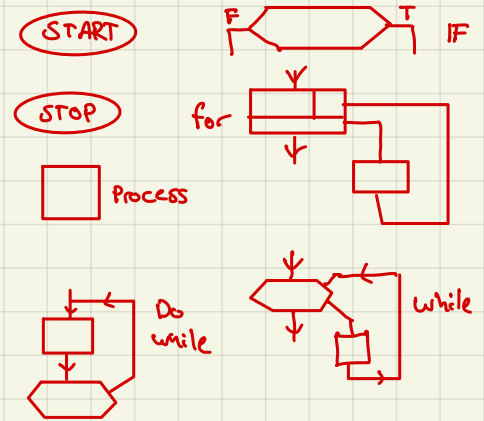
- * Sort
- * Search
- * Insert
- * Delete
- * Update

Sorting Algorithms

- * Bubble sort
- * Selection sort
- * Quick sort
- * Merge sort
- * Heap sort
- * Insertion sort
- * Shell sort
- * Radix sort
- * Counting sort (LSD, MSD) → Counting sort Algorithm

Comparison
Sort
Algorithms

Flowchart Symbols



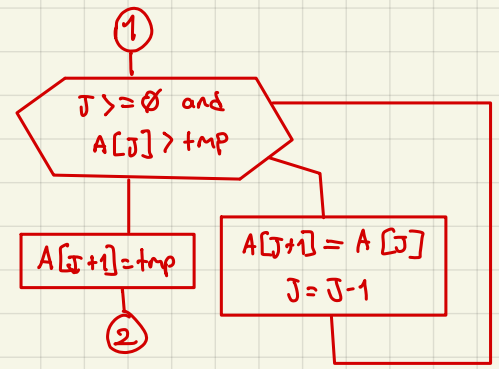
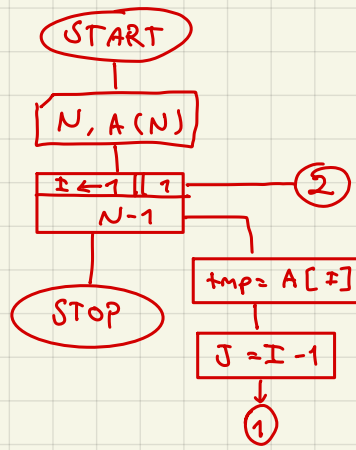
Insertion Sort

5 1 4 3 2 7 6

→ 1 5 4 3 2 7 6
 → 1 4 5 3 2 7 6
 → 1 3 4 5 2 7 6
 → 1 2 3 4 5 7 6
 → 1 2 3 4 5 6 7

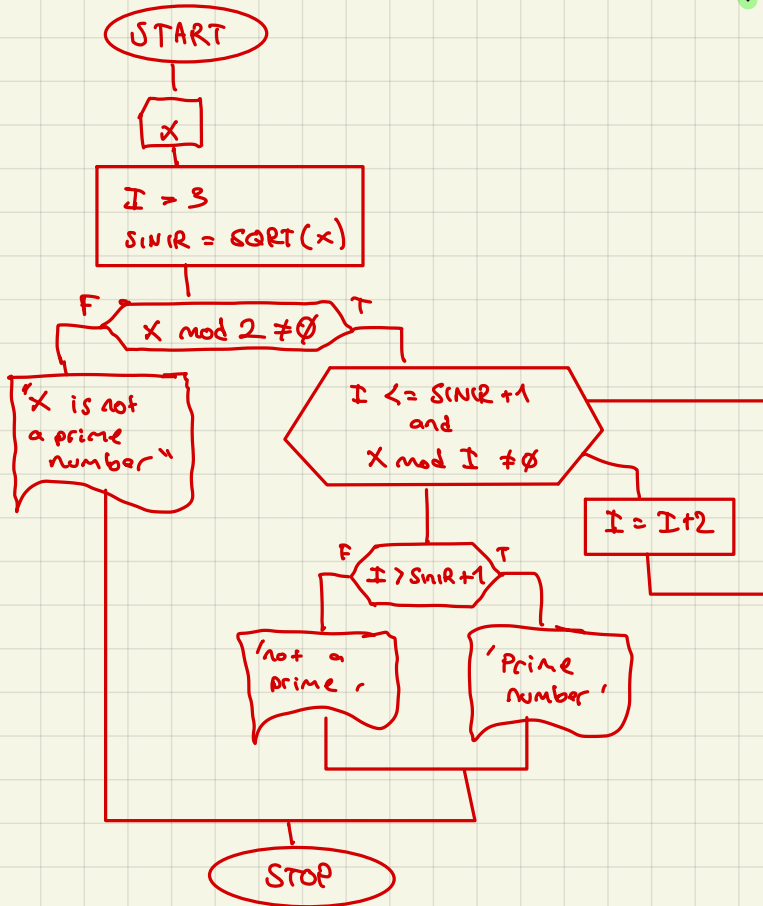
Worst-Case time complexity: $O(n^2)$

Best-Case time complexity: $O(n)$



Finding Prime Numbers

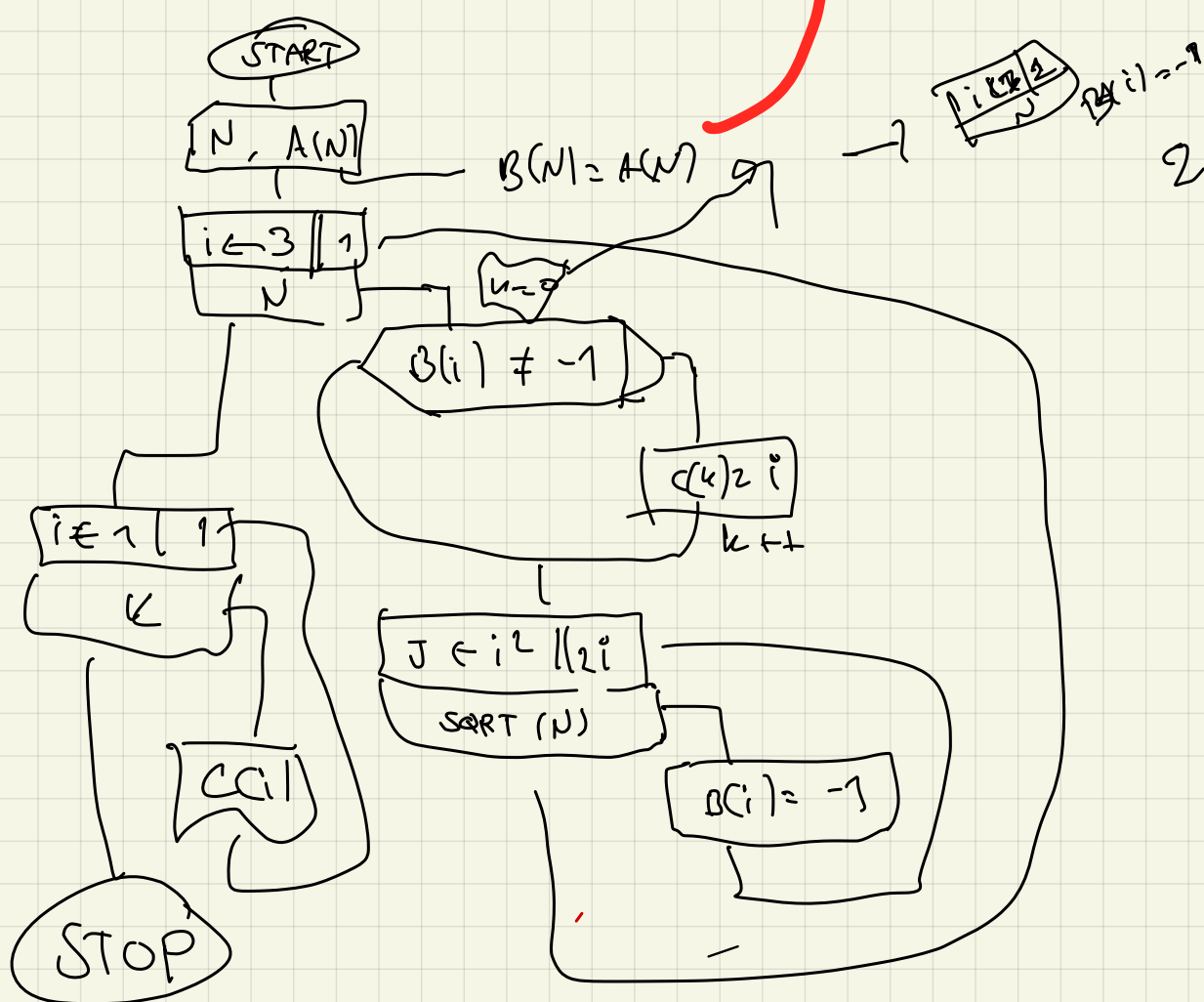
* Naive Solution (Brute force Approach) (asıl sayı bulurken sayının kareköküne kadar olan sayıları kontrol etmek asıl olup olmadığını anlamakta yeterlidir.)



Sieve of Eratosthenes Algorithm

her şeyi arastır, Akisimi Çiz gel ∇





Algorithm Analysis

- * Time complexity
- * Space Complexity

Basic Operations

- * Comparison Op.
- * Assignment Op.
- * Arithmetic Op.
- * Initialization Op.

Search algorithm bak !

Example: Find maximum value on array.

Pseudo Code:

Basic OP

```

2      current Max ← A[0]
2+n    For i ← 1 to n-1 do
2(n-1)   if A[i] > current Max then
2(n-1)     current Max ← A[i]
2(n-1)   increment counter
1      return current Max
  
```

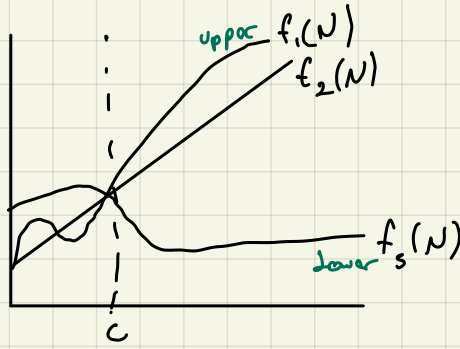
$$O(2n-1) \rightarrow O(n)$$

Asymptotic Notation

Big Oh (O) Notation upper bound

Big Theta (Θ) Notation Tight bound

Big Omega (Ω) Notation lower bound



Functions Growth Rate

1 $\log(N)$ N $N \log(N)$ N^2 N^3 2^N $N!$
 $\leftarrow$ ————— $\rightarrow$

1 5 8 4 3 → bu sayı dizisinden toplam olarak "13" ü bulmak için hangi sayıları seçmeniz gerektiğini bulan algoritmanın karmaşıklığı 2^N 'dir. Alt kümeleri kontrol etmeniz gerektiği için!

Recursive Bal

Recursive vs Iteration

Criteria

Iteration

Recursion

Recursive Pros and Cons

- İterasyona göre daha yavaş çalışır
- + Daha okunaklı ve az kodla yazılabilir
- + Bazı problemlerin algoritmik tasarımı daha kolaydır

- Made Of Implementation

Using Loops

function call itself

- State

Defined by the control variables value

Defined by the parameter values stored in stack

- Progression

The value of control variable moves toward the value in condition

The function state converges towards the base case

- Termination

Loop ends when control variable's value satisfies the condition

Recursion ends when
base case becomes true

- Code Size

Iterative code tends to be bigger in size

Recursion decreases the size of code

- No Termination State

Infinite Loops use CPU cycles

May cause stack overflow error or it might crash the system.

- Execution

faster

Slower

Factorial Program Using Recursion

```
int factorial (int n) {  
    if (n <= 1)  
        return 1;  
    return n * factorial (n-1);  
}
```

X^Y

10^3

10 10 10

```
int fonsiyon (int x, int y) {  
    if (y == 0)  
        return 1;  
    return x * fonsiyon (x, y-1);  
}
```

Verilen Sayı Kaç Basamaklı

```
int basamak (int x) {  
    if (x / 10 == 0)  
        return 1;  
    return 1 + basamak (x / 10);  
}
```

Opsiyon
Çözüm

```
int basamak (int x) {  
    static int count = 1;  
    if (x > 9) {  
        count++;  
        basamak (x / 10);  
    }  
    return count;  
}
```

Fibonacci Numbers → 0 1 1 2 3 5 8 13 21

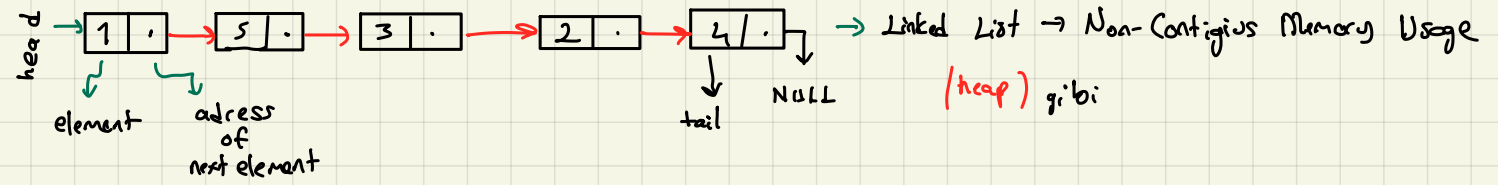
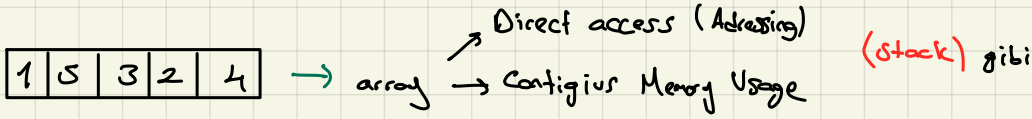
```
int fibonacci (int n) {  
    int a, b;  
    if (n <= 1)  
        return n;  
    a = 0, b = 1;  
    for (i = 2; i <= n; i++) {  
        c = a + b;  
        a = b;  
        b = c;  
    }  
}
```

Iterative

Recursive

```
int fibonacci (int n) {  
    if (n <= 1)  
        return n;  
    return fibonacci (n-1) + fibonacci (n-2);  
}
```

Linked Lists



* ekleme ve silme işleminin yoğun olduğu ve bellekte parça parça boşlukların olduğu durumlardan linked list'i kullanmak akıllıcadır.

* Cift array ya da n x 2 lik matrisler ile linkli liste kullanımı yapılabilir.

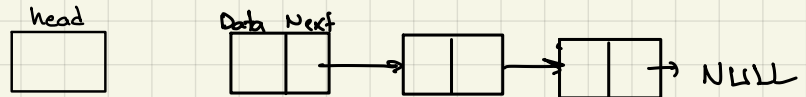
Type of Linked Lists

1. Single Linked Lists

2. Doubly Linked List

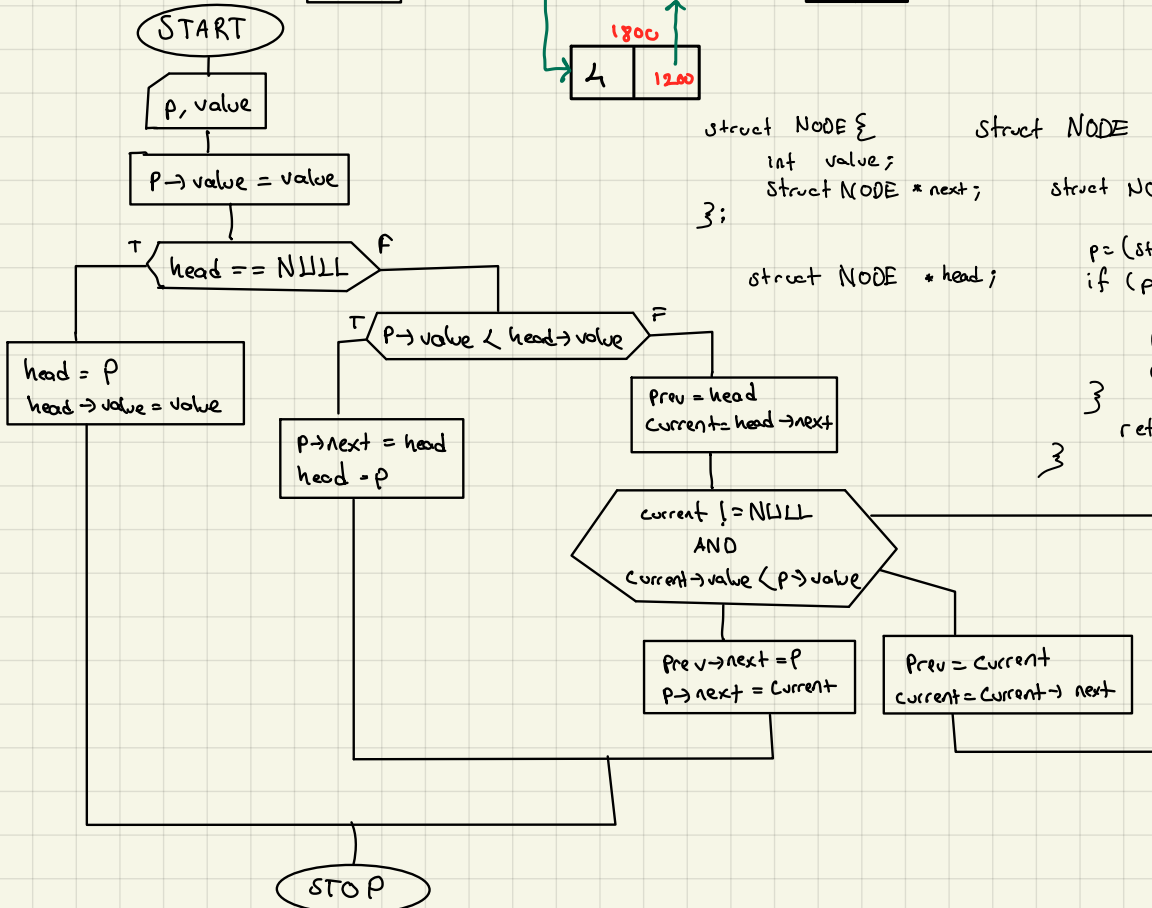
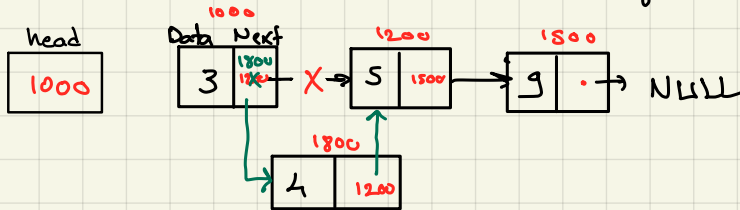
3. Circular Linked List

4. Circular Doubly Linked List



Insert Operation

* Sırasız bir linkli listede yeni eleman en başa koyulur.



```
struct NODE {  
    int value;  
    struct NODE * next;  
};  
struct NODE * head;
```

```
struct NODE * createNode() {
```

```
    struct NODE * p;
```

```
    p = (struct NODE *) malloc (sizeof (NODE));
```

```
    if (p == NULL) {
```

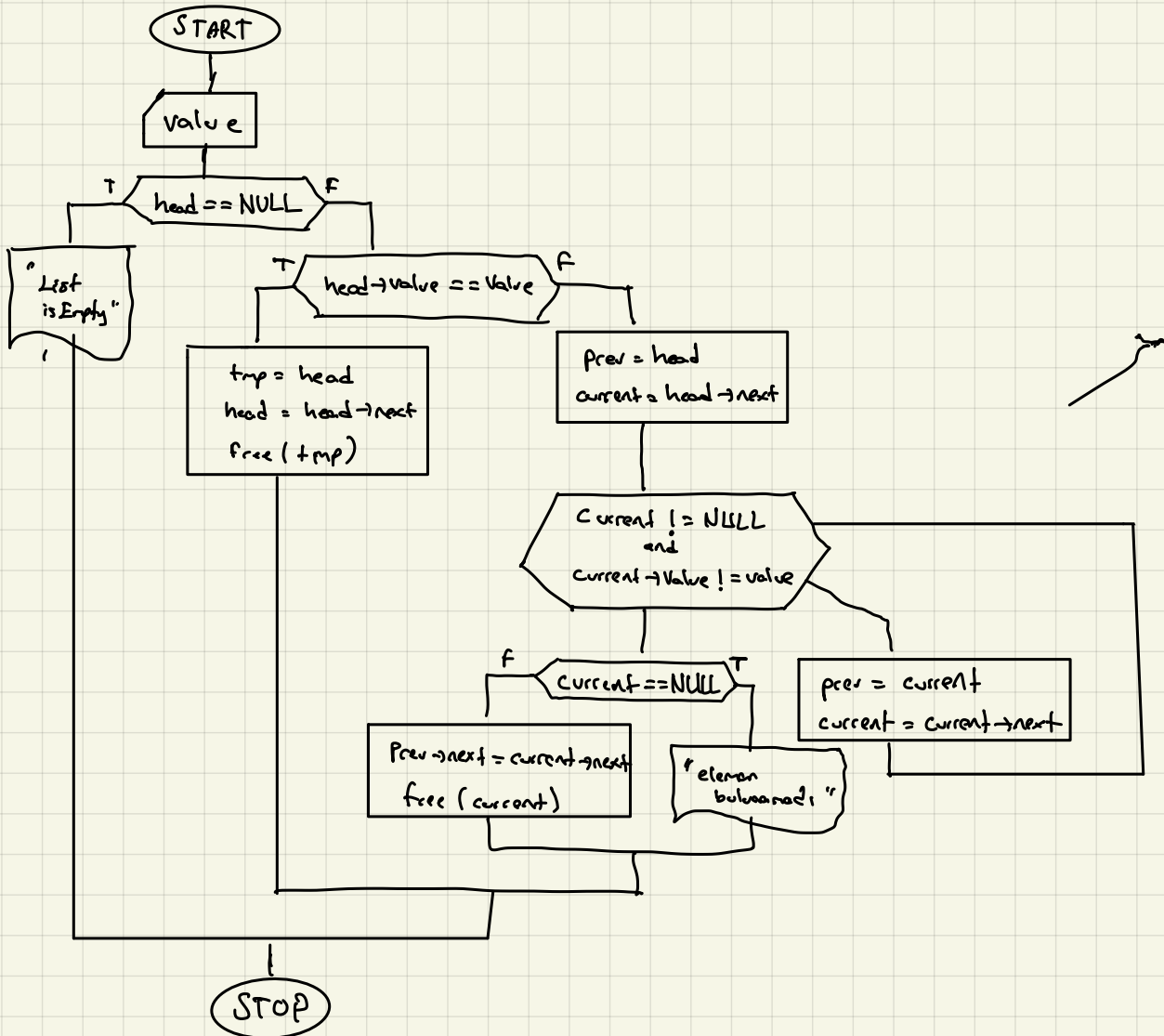
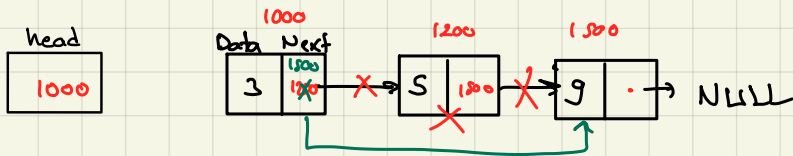
```
        printf ("Allocation error");
```

```
        exit (0);
```

```
    }  
    return p;
```

```
}
```

Delete Operations



Linked List Operations

- Traversal
- Insertion
- Deletion
- Search
- Update
- Sort

Linked list vs Array

Array

- Random Access

- Contiguous Memory usage

- Slower on insertion deletion

- Static Memory Allocation

- Single, Two, multi dimension

- Stack

- Cache Locality



bellekte belli kısım cache'e alındığı için

linked listde değişim olduğu için array avantajlı

Linked List

- Sequential Access

- Non-Contiguous Memory usage

- faster on insertion deletion

- Dynamic Memory Allocation

- Single, Doubly, Circular

- Heap

APPLICATIONS on LINKED LISTS

Real-Life

1-) Stack, Queue

2-) Graph

3-) Hashing

4-) Dynamic Memory Allocation

5-) Maintaining Directory of Names

6-) Representing Sparse Matrices

* Image viewer

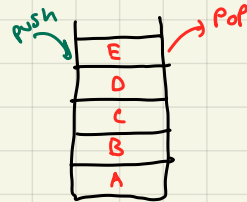
* Previous and next page in web browser

* Music player

* Operating System (Process Management)

STACK (yığın) (Abstract Data Type)

- first in last out **FILO**
- Last in first out **LIFO**



Applications

1 Evaluation of arithmetic Op.

$$A + (B - C (D * E / G) - F)$$

2 Recursion function usage

3 Reversing String

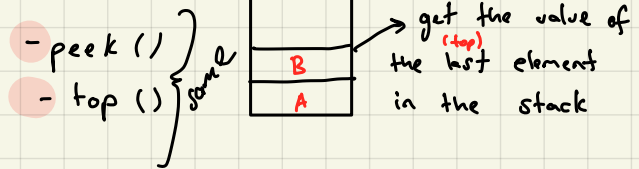
4 Backtracking (Depth First Search)

5 Parenthesis Matching

6 Conversion of Decimal to other Number Systems

Stack Operations

- push()
- pop()
- is Full()
- is Empty()

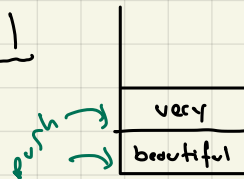


Stack Usage

The weather is very beautiful

The weather is very

The weather is



Stack Usage In Compilers

- | | | | |
|---|------------------|-------|----------------|
| 1 | - (a + b (| error | } Syntax Error |
| 2 |) a + b (| error | |
| 3 | a * (b + c (| error | |
| 4 | ((a + b) * c) | ✓ | |

Stack Operations

```
#define MAX 10
typedef struct {
```

```
    int item[MAX];
```

```
    int top;
```

```
} STACK;
```

```
void initStack (STACK *s) {
    s->top = 0;
```

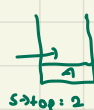
```
int is Empty ( STACK *s ) {
    if ( s->top == 0 )
        return 1;
```

```
    else
        return 0;
}
```

```
3
```

```
int is full ( STACK *s ) {
    if ( s->top == MAX )
        return 1;
    else
        return 0;
}
```

```
3
```



Stackte bir Sonrakı
nokta gösterildiği için azalış
pop edilir

```
int push ( int x, STACK *s ) {
    if ( isfull (s) )
        return 0;
```

```
    else {
        s->item [ s->top ] = x;
        ++s->top;
        return 1;
    }
```

```
3 int pop ( STACK *s, int *x ) {
    if ( is Empty (s) )
        return 0;
    else {
        --s->top;
        *x = s->item [ s->top ];
        return 1;
    }
```

```
3
```

```

int top (STACK *s, int *x) {
    int adr;
    if (Is Empty (s))
        return 0;
    else {
        adr = s->top - 1;
        *x = s->item [adr];
        return 1;
    }
}

```

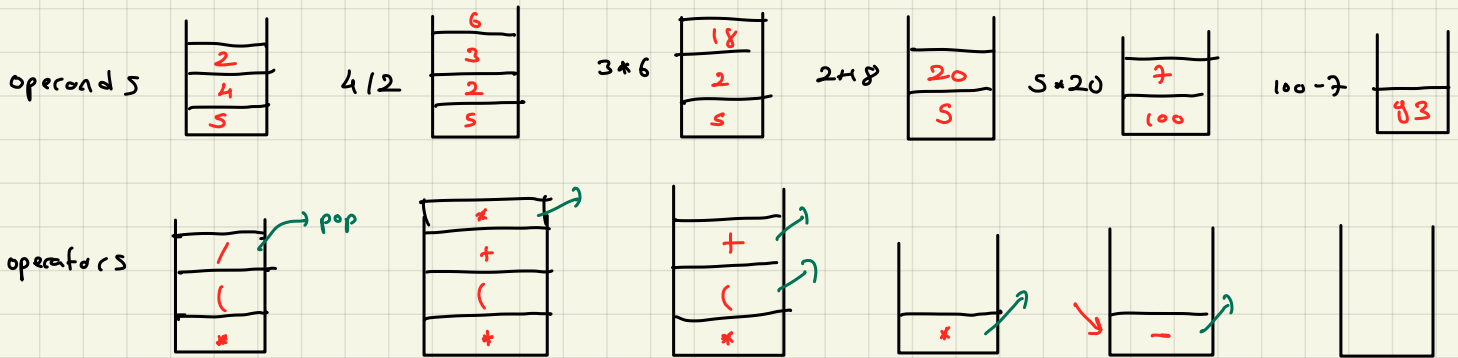
Arithmetic Expression Evolution

- Infix $\rightarrow A+B$
- Prefix $\rightarrow +AB$
- Postfix $\rightarrow AB+$

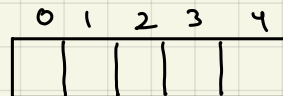
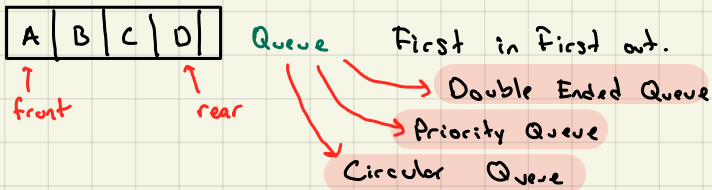
Precedence

(
* /
+ -

$$5 \times (4 / 2 + 3 * 6) - 7$$



Queue (kuyruk) (Abstract Data Type) iki uclu STACK



↑↑
first rear



↑ first ↑ rear



↑ first

↑ rear

→ Implementasyonu göre baş alanı gösterebilir.

Queue Operations

- * is Empty ()
- * is Full ()
- * enqueue ()
- * dequeue ()
- * initQueue ()
- * peek ()

* ?

Queue Implementation on Linked Lists

```
int isEmpty() {
    if (front == -1)
        return 1;
    else
        return 0;
}
```

```
int isFull() {
    if (rear == Max-1)
        return 1;
    else
        return 0;
}
```

```
int dequeue() {
    if (isEmpty())
        return 0;
    else {
        *X = q[front++];
    }
    return 1;
}
```

```
int enqueue(int value) {
    if (isFull())
        return 0;
    else {
        rear++;
        q[rear] = value;
    }
    return 1;
}
```

```
void initQueue() {
    int front = -1, rear = -1;
}
```

```
int isFull() {
    if ((front == rear + 1) || (front == 0 && rear == SIZE - 1))
        return 1;
    else
        return 0;
}
```

```
int enqueue(int value) {
    if (isFull()) {
        return 0;
    }
    else if (front == -1)
        front = 0;
    rear = (rear + 1) % SIZE;
    q[rear] = value;
    return 1;
}
```

```
int dequeue(int *X) {
    if (isEmpty())
        return 0;
    else {
        *X = q[front];
        if (front == rear) {
            front = -1;
            rear = -1;
        }
        else
            front = (front + 1) % SIZE;
        return 1;
    }
}
```

Double Ended Queue

- * Input Restricted dequeue (D.E.Q.)
input restricted at a single end but allows deletion at both the ends
- * Output Restricted Deque
output is restricted at a single end but allows insertion at both the ends

Comparison STACK vs Queue

	<u>STACK</u>	<u>Queue</u>
* Working Principle	LIFO	FIFO
* Structure	Same end used to insert and delete	One end used for insert Another end used for deletion
* Number of Pointers	one	two
* Operations	Push, Pop	Enqueue, Dequeue
* Variants	It does not have	Circular, Priority, Double Ended Queue
* Implementation	Simpler	Comparatively complex

Linear Data Structure

- * Array
- * Linked List
- * STACK
- * Queue

The data items are arranged in an orderly manner where the elements are attached adjacently

Simpler

Single

Non-Linear Data Structure

(Hierarchical Data Structure)

- * Graphs
- * Trees

It arranges the data in a sorted order and there exists a relationship between the data elements

Complex

Multiple

Terminology

- * Path
- * Root
- * Parent
- * Child
- * Leaf
- * Subtree
- * Visiting
- * Traversing
- * Level
- * Forest
- * Depth
- * Height

Basic

Ease of Implementation

Levels involved

Tree Data Structure

Root to leaf

Tree App. in Real-life

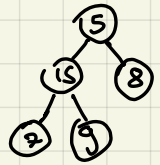
- File management system
- Compression Algorithms (Huffman Alg.)
- Databases
- Compilers
- Priority Queues
- AI (Decision Tree, Random forest)
- Indexing multi-dimensional information such as geographical coordinates, rectangles or polygons.

Types of Tree

- Binary tree
- Binary Search tree
- AVL tree
- B-tree
- R-tree
- Interval tree
- D-ary heap
- Heap tree

Tree Traversal

- Inorder Traversal left subtree $\rightarrow$ node $\rightarrow$ right subtree $7 \rightarrow 15 \rightarrow 9 \rightarrow 5 \rightarrow 8$
- Preorder Traversal node $\rightarrow$ left subtree $\rightarrow$ right subtree $5 \rightarrow 15 \rightarrow 7 \rightarrow 9 \rightarrow 8$
- Postorder Traversal left subtree $\rightarrow$ right subtree $\rightarrow$ node $7 \rightarrow 9 \rightarrow 15 \rightarrow 8 \rightarrow 5$

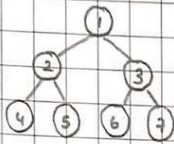


```
struct node {  
    int data;  
    struct node * left;  
    struct node * right;  
}
```

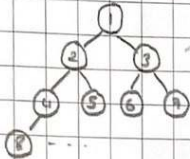
```
void inorderTraversal (struct node * root) {  
    If (root == NULL)  
        return;  
    inorderTraversal (root  $\rightarrow$  left);  
    printf ("%d", root  $\rightarrow$  data);  
    inorderTraversal (root  $\rightarrow$  right);  
}
```

```
void preOrderTraversal (struct node * root) {  
    If (root == NULL)  
        return;  
    printf ("%d", root  $\rightarrow$  data);  
    preOrderTraversal (root  $\rightarrow$  left);  
    preOrderTraversal (root  $\rightarrow$  right);  
}
```

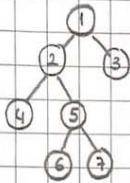
```
void postOrderTraversal (struct node * root) {  
    If (root == NULL)  
        return;  
    postOrderTraversal (root  $\rightarrow$  left);  
    postOrderTraversal (root  $\rightarrow$  right);  
    printf ("%d", root  $\rightarrow$  data);  
}
```

Perfect Binary Tree

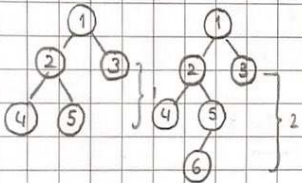
It is a type of binary tree which every internal node has exactly two child nodes and all the leaf nodes are at the same level

Complete Binary Tree

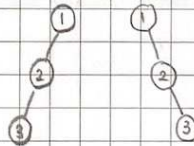
It is a binary tree in which all the levels are completely filled except possibly the lowest one, which is filled from the left

Full Binary Tree

It is a binary tree in which every parent/internal node has two or no children

Balanced Binary Tree

Uç noktalar arası birden fazla olmayacak

Degenerated or Pathological Tree**Skewed Binary Tree**

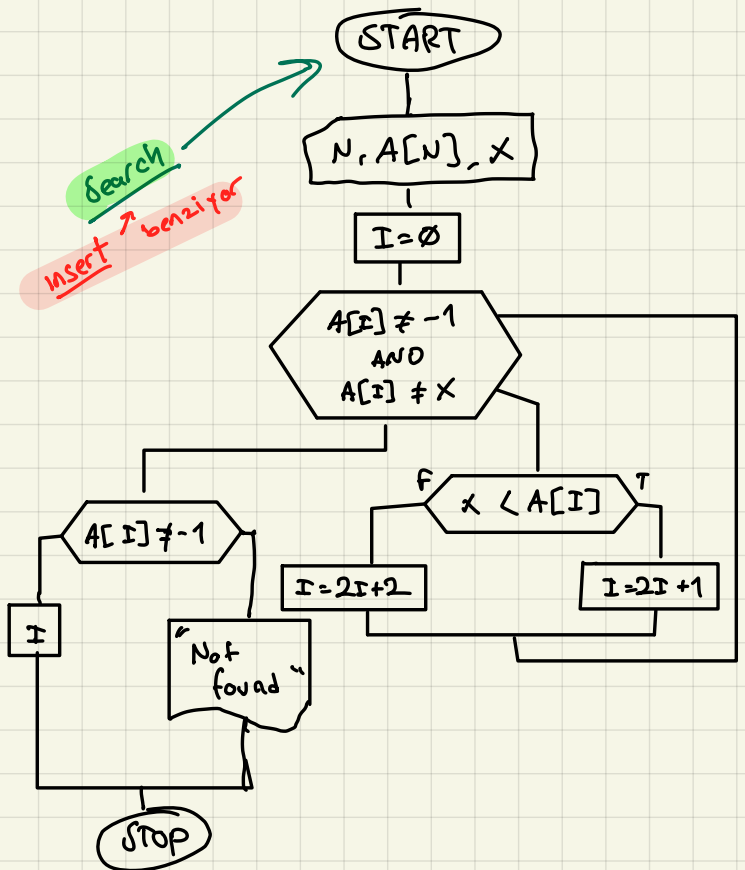
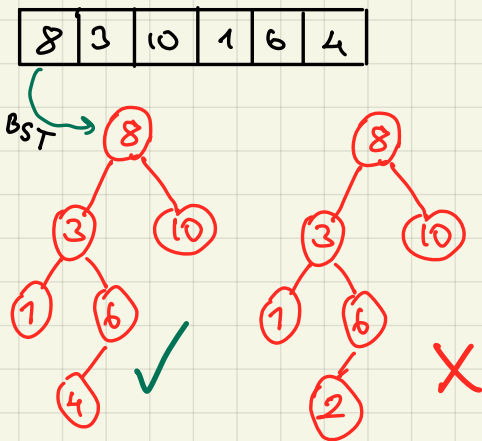
Binary Search Tree

- Search

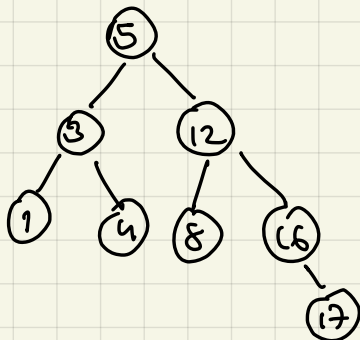
- Insert

- Delete

$\emptyset$ 1
 $2k+1$ $2k$
 $2k+2$ $2k+1$



Delete



Case I = Delete Leaf node

Case II = Delete internal node with a single child.

Case III = Delete internal node (right, left child).

Case IV = Not found

Case III:

- 1-) Get the inorder successor of that node
- 2-) Replace the node with the inorder successor
- 3-) Remove the inorder successor from its original position.

Inorder successor $\rightarrow$ kendisinden sonra gelen en büyük eleman

Inorder successor of a node is the next node in inorder traversal of a Binary Tree

Inorder successor of an input node can be also defined as the node with the smallest key greater than the key of the input node

- 1-) IF the right subtree of a node is not NULL, then successor lies in the right subtree.
- 2-) IF " " " " " " is Null, start from root;
 travel down the tree, if a node's data is greater than root's data then go right side, otherwise go left side.

```

struct node * deleteNode ( struct node * root, int key) {
    if ( root == NULL ) return root ;

    if ( key < root->key )
        root->left = deleteNode ( root->left , key );
    else if ( key > root->key )
        root->right = deleteNode ( root->right , key );
    else {
        if ( root->left == NULL ) {
            struct node * temp = root->right ;
            free ( root );
            return temp ;
        }
        else if ( root->right == NULL ) {
            struct node * temp = root->left ;
            free ( root );
            return temp ;
        }
        struct node * temp = minValueNode ( root->right );
        root->key = temp->key ;
        root->right = deleteNode ( root->right , temp->key );
    }
    return root ;
}

```

```

struct node * minValueNode ( struct node * node) {
    struct node * current = node ;
    while ( current && current->left != NULL )
        current = current->left ;
    return current ;
}

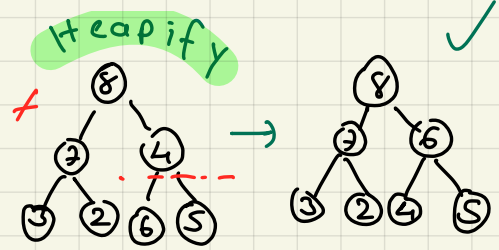
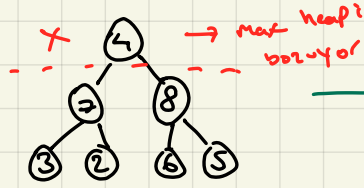
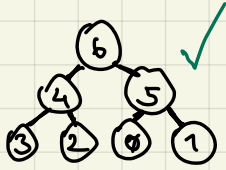
```

Heap Tree

- Complete Binary Tree $\rightarrow$?
- Max-Heap $\rightarrow A[\text{parent}(i)] \geq A[i]$
- Min-Heap $\rightarrow A[\text{parent}(i)] \leq A[i]$

Array :

6	4	5	3	2	0	1
---	---	---	---	---	---	---

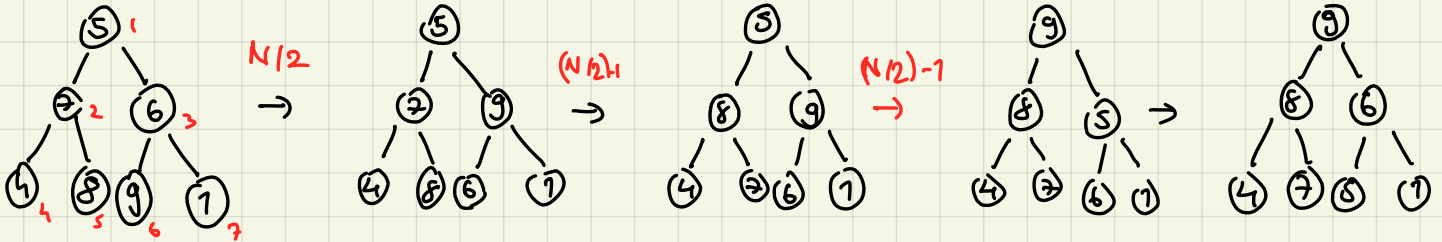


- Insert $\rightarrow$ sona ekle ve yine heapify yap

- Search

- Delete $\rightarrow$ Silmek istediğin node'u sona at, size'ı 1 düşür heapify yap

Max-Heap Tree Conversion



```
void buildMaxHeap(int dizi[]) {
    int i;
    for (i = N/2; i > 1; i--)
        maxHeapify(dizi, i, N);
}
```

3

Heap Sort
kendin bak

```
void maxHeapify(int dizi[], int i, int N) {
```

```
    int left = 2*i;
    int right = 2*i + 1;
    if ((left <= N) && dizi[left] > dizi[i])
        largest = left;
```

else

```
    largest = i;
    if ((right <= N) && dizi[right] > dizi[largest])
        largest = right;
    if (largest != i) {
```

```
        swap(dizi[i], dizi[largest]);
        maxHeapify(dizi, largest, N);
    }
```

3

Haftaya Priority
Queue

Delete
Heapify
bak

Heap Sort:

```
void heapSort(int arr[], int n) {
    for (i = n/2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    for (i = n - 1; i >= 0; i--) {
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }
}
```

```
void swap(int *a, int *b) {
    int temp;
    temp = *a;
    *a = *b;
    *b = temp;
}
```

Heap Sort Complexity

Best $O(n \log n)$
 Average $O(n \log n)$
 Worst $O(n \log n)$
 Space $O(1)$
 Stability NO

eger 2 aynı değer varsa
 ve bunlar sıralanırken sonra
 yerleri değişiyorsa Stable değildir.

#16 ahmet
 #17 ahmet
 Non Stable

Priority Queue

Linked List
 Array
 Heap Tree
 Binary Search Tree

Process Management and Interrupt Handling
 Dijkstra Algorithm
 Data Compression (Huffman Coding)
 Queue on Banking Operations
 Airplane Queues

Priority Queue Operations

- Insert
- Delete
- Peek

Operation Complexity

	Peek	Insert	Delete
• Linked list	$O(1)$	$O(n)$	$O(1)$
• Binary Tree	$O(1)$	$O(\log N)$	$O(\log N)$
• BST	$O(1)$	$O(\log N)$	$O(\log N)$

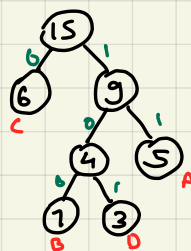
Priority Queue Using Heap Tree

Binary tree yerine heap tree tercih edilir
 Complexity açısından aynıdır.

Huffman Coding

B C A A D D D C C A C A C A C

1 6 5 3 → 1 3 5 6
 B C A D
 1 3 4 5 6 9
 1 3 4 5 6 9 15



C	6	x 6	6
A	11	x 5	10
B	10	x 1	3
D	10	x 3	9
			28

Algorithm

1. Calculate the frequency of each char in the string.
2. Sort the chars in increasing order of the frequency.
3. Make each unique char as leaf node.
4. Create an empty node n. Assign the min frequency to the left child of n and assign the second minimum frequency to the right child of n. Set the value of the n as the same of the above two min frequencies.

5. Remove these two min frequencies from Q and add the sum into the list of frequencies.
6. Insert node n into the tree.
7. Repeat steps 3 to 5 for all characters.

Example:

A → 5	1100	→ 20
B → 9	1101	→ 36
C → 12	100	→ 36
D → 13	101	→ 39
E → 16	111	→ 48
F → 45	0	→ + 45
		224

5 9 12 13 16 45
 5 9 12 13 14 16 45
 5 9 12 13 14 16 25 45
 5 9 12 13 14 16 25 30 45
 5 9 12 13 14 16 25 30 45 55 → 100

